Feedword Neural Nets

this week's pset

Recall log-linear models

$$p(y|x) = \frac{e^{v \cdot f(x,y)}}{\sum_{y' \in \mathcal{Y}} e^{v \cdot f(x,y')}}$$

$\mathcal{Y}$ = all possible outputs

parameters (learnable, using gradient descent)

feature vectors fixed / designed by hand

normally, binary vector features

Idea: learn $f(x,y)$, bc designing by hand hard

Step 1 Slightly modify form of log linear models

$$p(y|x) = \frac{e^{\vec{v}(y) \cdot \vec{f}(x) + \gamma_y}}{\cdots}$$

$$\sum_{y' \in \mathcal{Y}} e^{v(y') \cdot f(x) + \gamma_{y'}}$$

$\vec{v}(y)$, for every y, parameter vector of output y

$\vec{f}(x)$ = feature vector of input x

γ_y = "bias parameter" of output y (for every y)

Claim: this is just special case of log linear models

Pf: Ignore γ_y

$$\frac{e^{v(y) \cdot f(x)}}{\sum_{y' \in \mathcal{Y}} e^{v(y') \cdot f(x)}}$$

$|\mathcal{Y}| = m$, all possible outputs

$\mathcal{Y} = \{y_1, \ldots, y_m\}$

want $v \cdot f(x, y)$

$v(y) \cdot f(x)$ ($\in \mathbb{R}^d$, $\in \mathbb{R}^d$)

$\downarrow$

$v \cdot f(x, y)$

Define $f_i(x, y) = \begin{cases} f(x) & \text{if } y = y_i \\ \rightarrow & \downarrow \end{cases}$

$$f(x,y) := \begin{bmatrix} f_1(x,y) \\ \vdots \\ f_m(x,y) \end{bmatrix}$$

$|Y| = m$ copies of the d-dimensional $f(x)$ (but most 0ed out)

$$V = \begin{bmatrix} v(y_1) \\ \vdots \\ v(y_m) \end{bmatrix}$$

$m \times d$ dimension

$$v \cdot f(x,y) = \sum_{i=1}^{m} v(y_i) \cdot f_i(x,y)$$

all are 0 except for one s.t. $y_i = y$

$$\hookrightarrow = v(y) \cdot f(x)$$

We had γ_y

$$v \cdot f(x,y) = v(y) \cdot f(x) + \gamma_y$$

$$\begin{bmatrix} v(y_1) \\ \vdots \\ v(y_m) \end{bmatrix} \begin{bmatrix} f(x,y) \\ \vdots \end{bmatrix}$$

$$\begin{pmatrix} V(y_1) \\ \vdots \\ V(y_m) \\ \gamma_{y_1} \\ \vdots \\ \gamma_{y_m} \end{pmatrix} \cdot \begin{pmatrix} \vdots \\ f_m(x,y) \\ f_i'(x,y) \\ \vdots \\ f_m'(x,y) \end{pmatrix}$$

$$f_i'(x,y) = \begin{cases} 1 & y = y_i \\ 0 & \end{cases}$$

Next, we generalize / replace $f(x)$ with $\phi(x;\theta)$ ← parameters of the feature vector, makes it learnable

Def Feedforward model:

1. $\mathcal{X}$ = set of inputs — e.g. sentences over vocab V, $\mathcal{X} = V^M$, $y = V$
2. $\mathcal{Y}$ = set of outputs / labels, finite
3. $m > 0$ number of features = "representation dimension"
4. θ = parameters of

(4) θ parameters of feedforward network

Ω = set of possible values of θ

(usually $\mathbb{R}^{|\theta|}$)

(5) function $f : \mathcal{X} \times \Omega \to \mathbb{R}^m$

$f(x, \theta) \mapsto \phi(x; \theta)$ "representation of x"

(6) for each $y \in \mathcal{Y}$, $v(y) \in \mathbb{R}^m$ representation of label y

and bias $\gamma_y \in \mathbb{R}$

learnable params

$$p(y \mid x; \theta, v, \gamma) = \frac{e^{v(y) \cdot \phi(x;\theta) + \gamma_y}}{\sum_{y' \in \mathcal{Y}} e^{v(y') \cdot \phi(x;\theta) + \gamma_{y'}}}$$

"given" / under these parameters

softmax, takes $\mathbb{R}$-valued "scores"

$v(y) \cdot \phi(x;\theta) + \gamma_y \in \mathbb{R}$

$\phi(x;\theta) = \phi_\theta(x)$

next time, model this using neural nets

Feedforward NNs, continued

What is $\phi(x; \theta)$ input representation of x given params θ

Single layer feedforward network!

First, map/replace (α = to, α = with) $x \in X$ α input vector $f(x) \in \mathbb{R}^D$

D "input dimension"

this initial mapping (or replacement) is usually hard-coded

example

Handwritten Digit Recognition

X = pictures of $n \in \{1, \ldots, 9\}$

$f(x)$ $(x \in X)$ = vector of pixel values of image

eg 16×16 grey scale pixel images

$D = 256$, each $f(x)_i \in \mathbb{R}$ encoding brightness of that pixel

Language

vocab $V = \{x_1, \ldots, x_{|V|}\}$

OneHot $D = |V|$

each word mapped to $f(x) = [0 \ldots 0\ 1\ 0 \ldots 0] \in \mathbb{R}^D$ ($x = x_i$)

what about for x = sentence

$x = w_1 \cdots w_n$, fixed length n

$f(x)$ = concatenation of $f(w_i)$ = $\begin{bmatrix} f(w_1) \\ \vdots \\ f(w_n) \end{bmatrix}$

$\begin{bmatrix} \text{ONEHOT}(w_1) \\ \vdots \\ \text{ONEHOT}(w_n) \end{bmatrix}$

$D = |V| \cdot n$ huge. in practice this works

in language applications

word $w \in \mathbb{R}^{|V|}$ $\longrightarrow$ $\text{ONEHOT}(w)$ $\xrightarrow{\times E}$ $E \cdot \text{ONEHOT}(w) \in \mathbb{R}^{d \cdot |V|}$

E is learned, $E \subseteq \theta$, part of learnable params

$E : d \begin{pmatrix} & |V| & \end{pmatrix}$

$d < |V|$, gets dimension down

what about x = sentence

Many approaches

Simple approach 1: $\sum_{i=1}^{n} E \cdot \text{ONEHOT}(x_i) = E' \cdot f(x)$

$E' = (\underbrace{E \cdots E}_{n \text{ times}})$

Back to NN, assume we've replaced x w/ vector input representation (x, have $f(x) \in \mathbb{R}^D$ $f(x) = \text{ONEHOT}(x)$ or $E \cdot \text{ONEHOT}(x)$)

Neuron main ingredient for FFNNs

defined by ① weight vector $w \in \mathbb{R}^D$

+ ② bias $b \in \mathbb{R}$

+ ③ transfer function $g : \mathbb{R} \to \mathbb{R}$

defined by

$$h = g(\underbrace{w \cdot x + b}_{\text{input to neuron}})$$

h = neuron's output = "hidden"

g often called "non-linearity" bc g almost never linear

If g were linear (affine) $g(z) = \alpha \cdot z + \beta$, $\alpha, \beta \in \mathbb{R}$

Not interesting/useful bc

$$g(w \cdot f(x) + b) = \alpha(w \cdot f(x) + b) + \beta$$

$$= \underbrace{(\alpha w)}_{\in \mathbb{R}^D} \cdot f(x) + \underbrace{(\alpha b + \beta)}_{\in \mathbb{R}}$$

Call this w' ; b'

$$= w' \cdot f(x) + b'$$

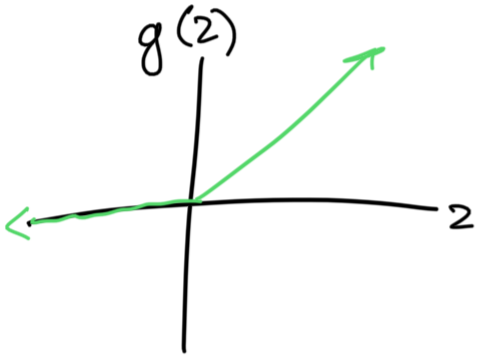
g adds nothing.

Common non-linear choices of g: ReLU, tanh

Def ReLU ("rectified linear unit")

$$g(z) = \begin{cases} z & \text{if } z \geq 0 \\ 0 & \text{otherwise } (z < 0) \end{cases}$$

$= \max\{0, z\}$



derivative?

$$\frac{dg(z)}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \\ \text{undefined at } z = 0 \end{cases}$$

g (ReLU) not discontinuous

g' ($\frac{dg}{dz}$) is discont. at one point

Def tanh

$$g(z) = \tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

Some intuition

as $z \to \infty$ clearly $e^{2z-1} \rightsquigarrow e^{2z+1}$

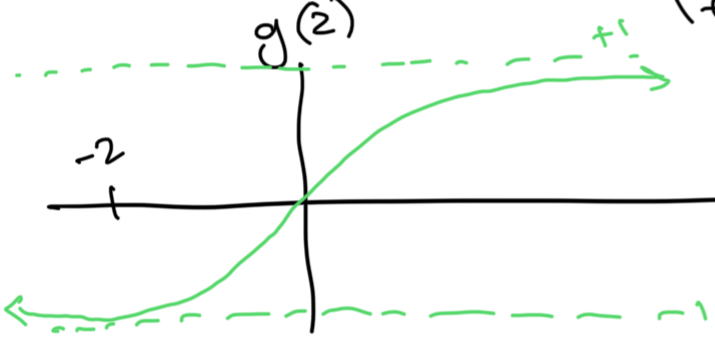
$g(z) \rightsquigarrow 1$ from below

as $z \to -\infty$ $\lim e^{2z} \to 0$

$\lim g(z) \to \frac{0-1}{0+1} \to -1$ from above

$\left|\frac{z-1}{z+1}\right| < -1$

at 0, $g(0) = \frac{-1}{1+1} = 0$



Derivative

$$\frac{dg(z)}{dz} = 1 - (g(z))^2$$

important property

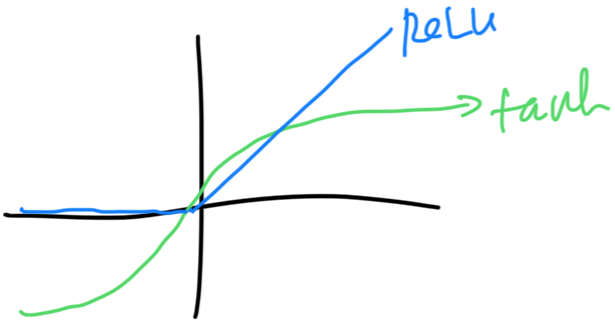
antisymmetric

$g(-z) = -g(z)$ ...

easy to show

$$g(-z) = \frac{\left(e^{-2z}-1\right) \times e^{2z}}{\left(e^{-2z}+1\right) \times e^{2z}} = \frac{e^{0}-e^{2z}}{e^{0}+e^{2z}} = -\frac{\left(e^{2z}-1\right)}{e^{2z}+1}$$

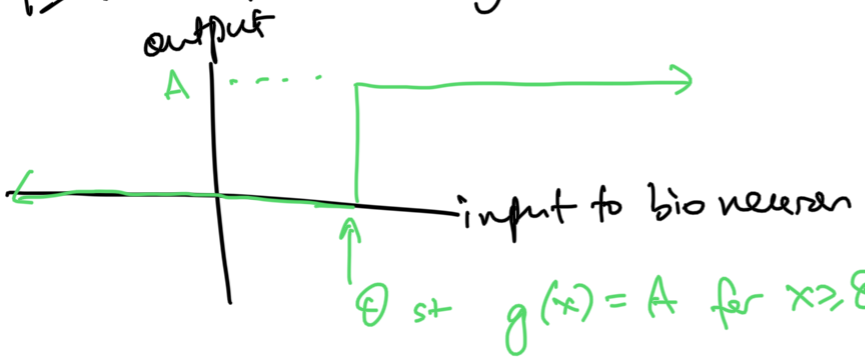
$$= -g(z)$$



both monotone, never decreasing in argument

$x' > x \quad g(x') \geq g(x)$

Based off biological neuron



θ s.t. $g(x) = A$ for $x \geq \theta$, #threshold

ReLU neuron in geometric point of view

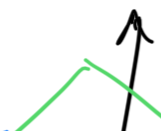
$$\text{ReLU}(w \cdot x + b) \equiv \begin{cases} w \cdot x + b & \text{for } wx + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$w \cdot x + b \geq 0$

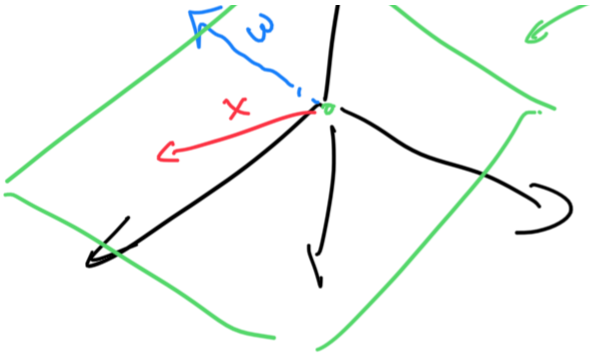
ignore ↑

$w \cdot x \geq 0$

$D = 3$



hyperplane of vectors v



s.t. $w \cdot v = 0$

in $D=3$, for any $w \in \mathbb{R}^3$ set of $\{v : w \cdot v = 0\}$ has dim 2, plane

in $D>3$, "hyperplane" b.c. has dim $D-1$

for $x \in \mathbb{R}^D$

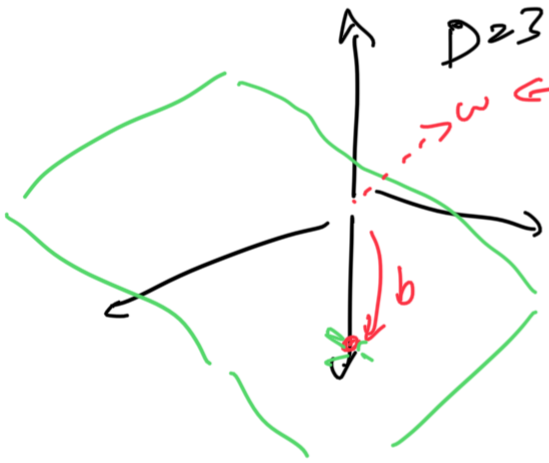
if $w \cdot x = 0$ or the hyperplane

$w \cdot x > 0$: x is on side of hyperplane that w is pointing in

$w \cdot x < 0$: x is on opposite side of hyperplane from where w is pointing

as $|w \cdot x| \to +\infty$, x has greater distance from hyperplane

$w \cdot x + b$: shifting hyperplane by b



hyper-plane

"normal vector"

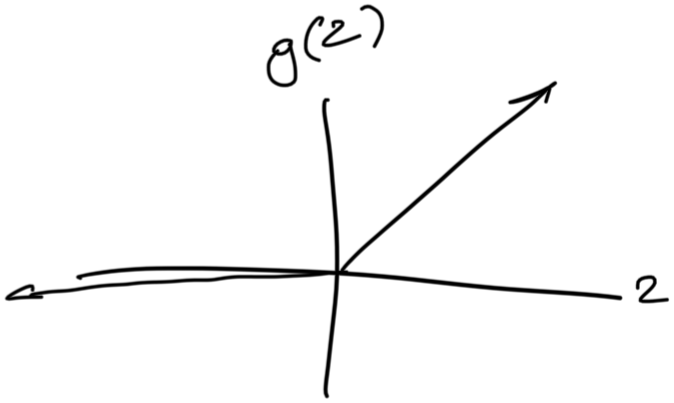
ReLU: zero out any input not on +ve side of hyperplane defined by w, b

tanh: also measures how much $f(x)$ is on the right side of a separating hyperplane, but

- keeps negative side "alive"
- diminishing output on each side of hyperplane

Rarely, might see other g eg "Leaky ReLU"

$$g(z) = \begin{cases} z & \text{if } z \geq 0 \\ 0.01 \cdot z & \text{for } z < 0 \end{cases}$$

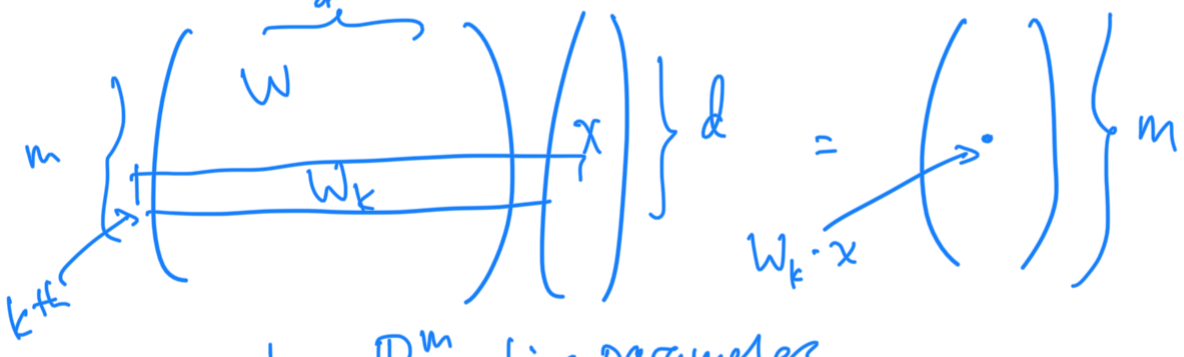


Def Single Feedforward Layer

(from now on, write x to mean $f(x)$, $x \in \mathbb{R}^D$)

parameters θ = matrix $W \in \mathbb{R}^{m \times D}$ m = # hidden neurons

$W_k \in \mathbb{R}^D$ = k-th row of W



$b \in \mathbb{R}^m$ bias parameter

k-th neuron input $z_k = W_k \cdot x + b_k$

k-th neuron output $h_k = g(z_k)$

overall, $\phi(x : \theta) = \begin{bmatrix} h_1 \\ \vdots \end{bmatrix} = \begin{bmatrix} g(W_1 x + b_1) \\ \vdots \end{bmatrix}$

$$\begin{bmatrix} h_m \end{bmatrix} \quad \begin{bmatrix} g(W_m \cdot x + b_m) \end{bmatrix}$$

parameters in θ?

$\theta = m \times D + m = m(D+1)$ real numbers

(Notation)

Generalize transfer func g to $\mathbb{R}^m \to \mathbb{R}^m$

how? apply entry-wise

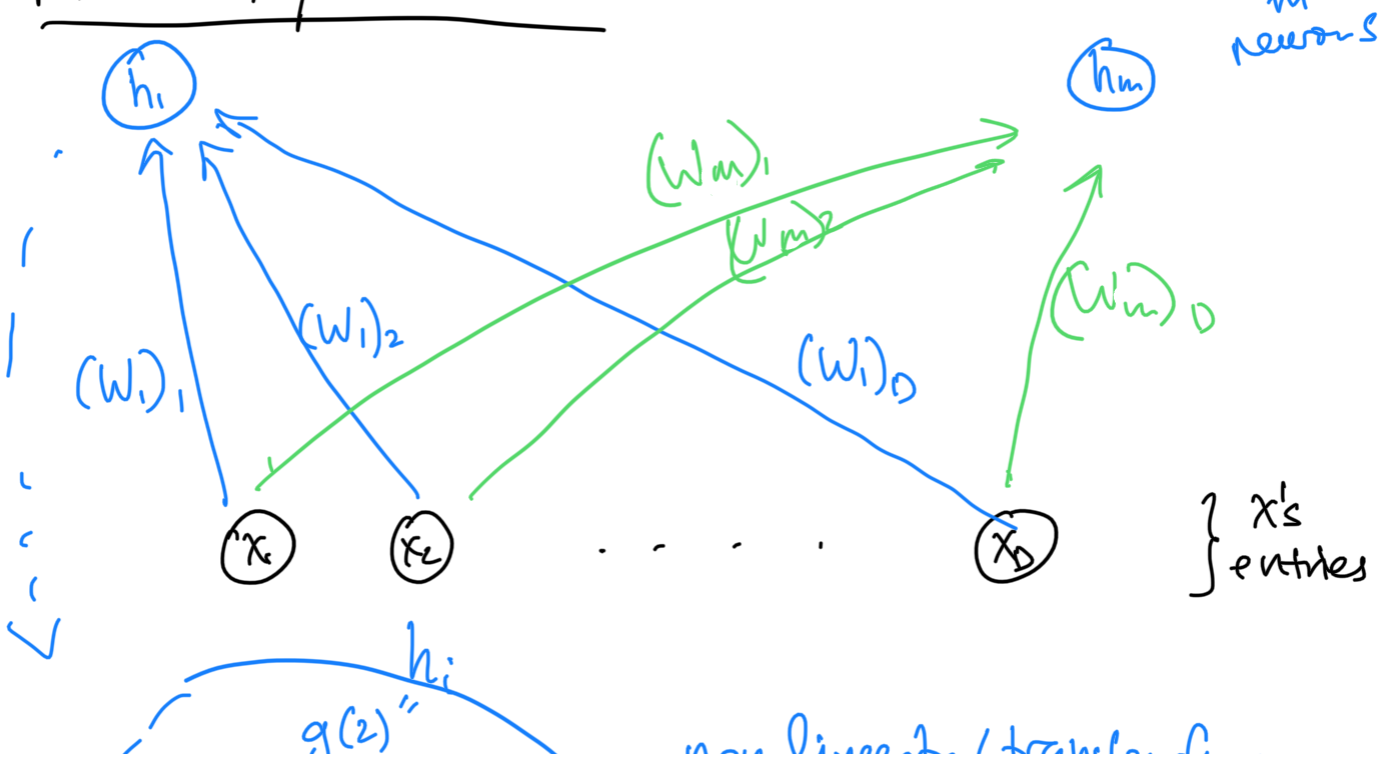
$$g\left(\begin{bmatrix} z_1 \\ \vdots \\ z_m \end{bmatrix}\right) = \begin{bmatrix} g(z_1) \\ \vdots \\ g(z_m) \end{bmatrix}$$

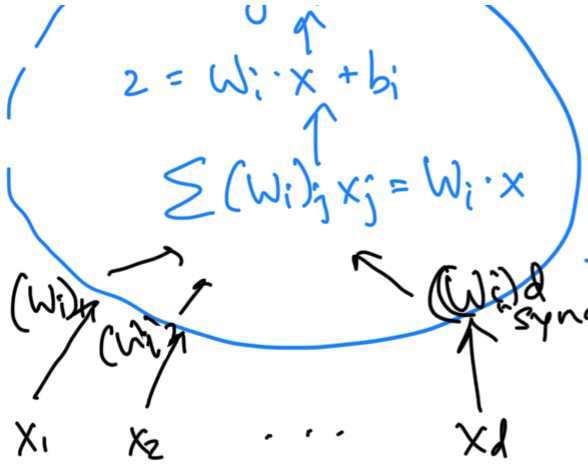
Can compactly write a FF layer

inputs to neuron $z = W \cdot x + b$

outputs of neurons $h = g(z)$

Picture representation





non-linearity / transfer fun

bias ("threshold") applied sum weighted inputs

"synapse weights", multiplied by inputs

XOR Problem

$y = \{-1, +1\}$

input $X = \{$

	y		y
$x^1 = [0,0]$	-1	$x^3 = [0,1]$	$+1$
$x^2 = [1,0]$	$+1$	$x^4 = [1,1]$	-1

$\}$

XOR = exclusive or, exactly one of the 2 inputs to XOR needs to be 1

Theorem 1 Assume g (transfer func) is linear/identity, and we use single layer ff network model,

i.e. $$p(y|x) = \frac{e^{v(y) \cdot x + \gamma_y}}{\sum_{y'} e^{v(y') \cdot x + \gamma_{y'}}}$$

the there are no parameters $\left(\begin{matrix} v(+1), v(-1) \\ \gamma_{+1}, \gamma_{-1} \end{matrix} \right)$

such that model correctly predicts XOR func

↳ = XOR if sampled using arg max

i.e. can't get it to output

$p(y|x) > 0.5$ for x, y as defined by XOR

Lemma for any x

$$p(+1|x) > 0.5$$

iff $u \cdot x + \gamma > 0$ where $u = v(+1) - v(-1)$

$$\gamma = \gamma_{+1} - \gamma_{-1}$$

and $p(-1|x) > 0.5 \iff u \cdot x + \gamma < 0$

Pf $$p(+1|x) = \frac{e^{v(+1)\cdot x + \gamma_{+1}}}{e^{v(+1)\cdot x+\gamma_{+1}} + e^{v(-1)\cdot x+\gamma_{-1}}}$$

$$= \frac{1}{e^{v(+1)\cdot x+\gamma_{+1}-(v(+1)\cdot x+\gamma_{+1})} + e^{v(-1)\cdot x+\gamma_{-1}-(v(+1)\cdot x+\gamma_{+1})}}$$

$$= \frac{1}{1+e^{(v(-1)-v(+1))\cdot x+(\gamma_{-1}-\gamma_{+1})}} = \frac{1}{1+e^{-u\cdot x-\gamma}}$$

$$= \frac{1}{1+e^{-(u\cdot x+\gamma)}}$$

$p(+1|x) > 0.5$

$$\iff \frac{1}{1+e^{-(u\cdot x+\gamma)}} > 0.5$$

$$2 > 1+e^{-(u\cdot x+\gamma)}$$

$$\iff e^{-(u\cdot x+\gamma)} < 1$$

$e^z < 1 \iff z < 0$

$$\iff -(u\cdot x+\gamma) < 0$$

$$\iff u\cdot x+\gamma > 0$$

Same proof for shows $p(-1|x) > 0.5$

$\iff v\cdot x+\gamma < 0$

Pf of Thm 1 using Lemma

Need x^1 $u \cdot [0,0] + \gamma < 0$

x^2 $u \cdot [0,1] + \gamma > 0$

x^3 $u [1,0] + \gamma > 0$

x^4 $u [1,1] + \gamma < 0$ $\quad -u[1,1] - \gamma > 0$

Derive Contradiction

$x^2 + x^3$ $\quad u \cdot \left([0,1] + [1,0] \right) + 2\gamma > 0$

$u \cdot [1,1] + 2\gamma > 0$

Subtract x^4 $\quad 2\gamma - \gamma > 0$

$\gamma > 0$

x^1 ~~$u \cdot [0,0] < 0$~~

$u \cdot [0,0] + \gamma = 0 + \gamma < 0$ ↯

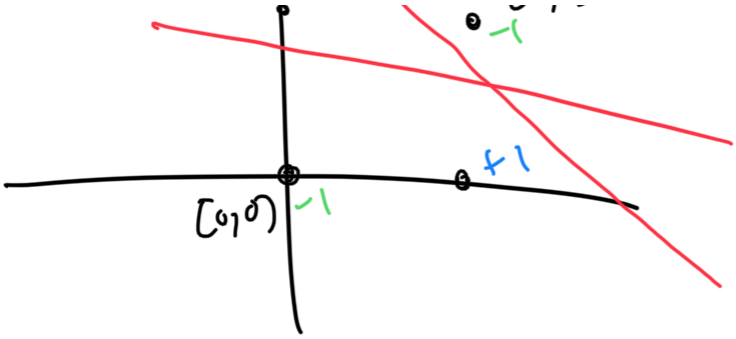
Geometric interpretation

Needed u, γ s.t. $\quad u \cdot x + \gamma > 0 \quad x \in [0,1), [1,0]$

$u \cdot x + \gamma < 0 \quad x = [0,0], [1,1]$

same as saying u is a normal vector of plane separating the x's: $[0,1], [1,0]$ are on +ve side of plane

$[0,0]$ $\quad [1,0]$



No way to place such a separating hyperplane